

# SBC — Skill de référence

Application desktop **Python / PySide6**, stockage **JSON + filesystem local**, macOS et Windows.  
Code source dans **/src**. Aucun compte en ligne requis. L'utilisateur est propriétaire de toutes ses données.

**Légende :**  implémenté ·  planifié / post-MVP

## Vision produit

Rassembler en un seul endroit tout ce qui compose un film IA personnel :  
projets, personnages, lieux, scénario, prompts et médias générés.  
Permettre de reprendre un projet après une pause sans perdre le contexte.  
Connecter à terme les outils IA externes (OpenArt, Veo, Gemini, ElevenLabs...)  
via un module bridge — l'architecture anticipée dès maintenant dans le modèle de données.

## Concept central — Tout est Block

Il n'existe qu'une seule entité business : le **Block**.

Un personnage est un Block. Un lieu est un Block. Un shot est un Block.  
Une image générée est un Block. Un prompt est un Block.  
Tout contenu dans l'application est un Block.

Un **projet** et une **bibliothèque** ne sont pas des Blocks — ce sont des **workspaces filesystem** (répertoires **.sbcprj**) qui contiennent des Blocks.  
La racine d'un projet est représentée *dans* le graphe par un Block  
**CONTAINER / profile="workspace\_root"**, mais le projet lui-même est le répertoire.

Le modèle unifié permet de réutiliser les mêmes composants UI (grille, arbre, inspecteur, carousel) et les mêmes services CRUD quelle que soit la sémantique métier du Block.

## Modèle de données — Block

Défini dans **src/domain/models.py**. Dataclass avec **slots=True**.

```
@dataclass(slots=True)
class Block:
    # --- Identité ---
    id: str                # "blk_<uuid_hex12>" ou ID racine hardcodé
    type: BlockType        # Nature physique/media
    profile: str            # Tag sémantique (voir PROFILES ci-dessous)
    name: str
    description: str = ""

    # --- Texte & Prompt ---
    prompt_ref: str = ""   # Prompt de référence (intention originale)
    prompt_generated: str = "" # Prompt final généré/enrichi
    comment: str = ""      # Note interne, non exposée à l'IA

    # --- Organisation ---
    shared: bool = False   # Δ DETTE – champ mort, voir ci-dessous
    domain: BlockDomain = BlockDomain.LIB
    functional_name: str = "" # Nom fonctionnel machine (slot, rôle...)
    tags: list[str]         # Tags libres

    # --- Accès & Provenance ---
    access_mode: BlockAccessMode = BlockAccessMode.OWNED
    provenance: dict[str, Any] # Voir section Provenance

    # --- Contenu media ---
    content: dict[str, Any]   # Dict libre – lire via as_media() / as_container()

    # --- Relations ---
    contains: list[str]       # IDs des Blocks enfants (CONTAINER seulement)
    inputs: list[InputConnection] # Connexions entrantes (liens typés par port)
    container_paths: dict[str, str] # {container_id → chemin virtuel dans FreeTree}
```

```
# --- Vues structurelles embarquées ---
tree: FreeTree | None = None # Vue hiérarchique (CONTAINER seulement)
graph: FreeGraph | None = None # Vue spatiale (CONTAINER seulement)
```

Méthodes sur Block

```
block.is_container() -> bool # type == CONTAINER
block.is_link()      -> bool # access_mode == LINK
block.is_editable()  -> bool # not is_link()

block.as_media()     -> MediaContent # Vue typée de content pour IMAGE/VIDEO/AUDIO
block.as_container() -> ContainerContent # Vue typée de content pour CONTAINER
```

⚠ Dette de refactoring — **shared: bool** est un champ mort

**block.shared** était prévu pour marquer les Blocks visibles depuis des workspaces externes. Ce concept a été supplanté par le système **Clone / Link** et le format de librairie unifié (**.sbcprj** identique pour projets et librairies).

État actuel :

- Toujours initialisé à **False** dans tous les sites de création (14 occurrences)
- **UseCaseService.list\_shared\_blocks()** retourne systématiquement une liste vide
- **ContainerResolver.shared\_contained\_blocks** est toujours vide
- Le champ apparaît encore dans **block\_properties\_editor.py** (UI) et dans **serialization.py** (persistance JSON) — mais sans effet fonctionnel

Le partage entre workspaces est désormais géré par :

- **access\_mode = LINK** → lecture seule, référence vers source externe
- **provenance.kind = "lib\_clone" | "lib\_link"** → traçabilité de l'origine
- Les librairies montées (**.sbcprj** de type **"library"**) → granularité workspace

Cible refactoring :

1. Retirer **shared** de **Block**, **BlockService**, **UseCaseService**, **ContainerResolver**
2. Retirer le champ de **block\_properties\_editor.py** (UI)
3. Conserver la lecture en désérialisation (**serialization.py**) avec **data.get("shared", False)** pour la compatibilité des fichiers existants — puis supprimer après migration
4. Supprimer **list\_shared\_blocks()** de **UseCaseService**

Enums

Définis dans **src/domain/models.py**. Tous héritent de **str**, **Enum** — sérialisables JSON via **.value**.

BlockType — Nature physique du Block

| Valeur      | Usage                                                |
|-------------|------------------------------------------------------|
| "empty"     | Placeholder, drop zone vide                          |
| "container" | Nœud structurel — peut contenir des enfants          |
| "image"     | Fichier image (PNG, JPG, WebP...)                    |
| "video"     | Fichier vidéo (MP4, MOV...)                          |
| "audio"     | Fichier audio (MP3, WAV, OGG...)                     |
| "text"      | Contenu textuel (note, dialogue, description...)     |
| "prompt"    | Prompt IA (texte destiné à être envoyé à un service) |

BlockDomain — Workspace fonctionnel

| Valeur       | Workspace        | Racine associée         | Statut    |
|--------------|------------------|-------------------------|-----------|
| "characters" | Personnages      | blk_characters_root     |           |
| "story"      | Scénario / Shots | blk_story_root          |           |
| "location"   | Lieux            | (voir dette ci-dessous) | ⚠ partiel |

| Valeur | Workspace          | Racine associée                | Statut |
|--------|--------------------|--------------------------------|--------|
| "lib"  | Librairie / Projet | blk_lib_root, blk_project_root | ✓      |

⚠ Dette de refactoring — BlockDomain.LOCATION sans infrastructure

BlockDomain.LOCATION est déclaré dans l'enum et le profil "location" existe, mais aucune infrastructure n'est en place : pas de root block dédié, pas de workspace panel, pas de service, pas de partition de stockage.

Comportement actuel (transitoire) :

Les Blocs de type Lieu utilisent domain=BlockDomain.CHARACTERS avec profile="location" et vivent dans blk\_characters\_root. Ils sont visibles dans le CharacterWorkspacePanel.

Ne pas créer de Blocs avec domain=BlockDomain.LOCATION — ils ne seraient rattachés à aucun workspace root et tomberaient dans le fallback project\_root, invisibles dans l'UI.

Cible refactoring — ce qu'il faudra créer :

```
LOCATION_ROOT_BLOCK_ID = "blk_location_root" # nouveau Block workspace_root
LocationWorkspacePanel # UI/Frames/workspaces/
LocationWorkspaceService # application/workspaces/
workspaces/location_root/blocks.json # nouvelle partition stockage
```

Une fois l'infrastructure en place, migrer les Blocs profile="location" existants vers domain=BlockDomain.LOCATION.

BlockAccessMode — Politique de mutabilité

| Valeur  | Signification                                 |
|---------|-----------------------------------------------|
| "owned" | Éditable localement                           |
| "link"  | Lecture seule — référence vers source externe |

BlockProvenanceKind — Origine du Block

| Valeur      | Signification                                        |
|-------------|------------------------------------------------------|
| "local"     | Créé dans ce projet/librairie                        |
| "lib_clone" | Copie locale d'un Block de librairie (indépendante)  |
| "lib_link"  | Référence vers un Block de librairie (lecture seule) |

Ces trois valeurs sont les seules présentes dans l'enum. Toute autre valeur lève un ValueError à la désérialisation.

PortType — Ports logiques pour InputConnection

| Valeur   | Usage                     |
|----------|---------------------------|
| "in"     | Entrée générique          |
| "top"    | Connexion bord supérieur  |
| "bottom" | Connexion bord inférieur  |
| "out"    | Sortie (rarement utilisé) |

Profils (PROFILES) ✓

Définition

Le profile est le croisement du type et de l'usage d'un Block.

- BlockType exprime la nature physique : qu'est-ce que c'est ?
- profile exprime le rôle métier : à quoi ça sert dans le projet ?
- Ensemble ils définissent complètement l'identité d'un Block.

```
BlockType.IMAGE + profile="reference" → une image qui sert de référence visuelle
BlockType.IMAGE + profile="generated" → une image produite par un service IA
BlockType.AUDIO + profile="voice" → un fichier audio qui est une voix de personnage
BlockType.CONTAINER + profile="shot" → un container qui structure un plan
```

Défini dans `src/domain/models.py` comme set ouvert — peut être étendu.

Combinaisons valides — BlockType x profile

| BlockType | Profils (usages) valides                                                                                                                              |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| CONTAINER | <code>workspace_root · container · shot · character · character_form · location · library · library_folder · template_slot · metadata · config</code> |
| IMAGE     | <code>asset · reference · generated · variation</code>                                                                                                |
| VIDEO     | <code>asset · generated · variation</code>                                                                                                            |
| AUDIO     | <code>voice · music · sfx · asset · generated</code>                                                                                                  |
| TEXT      | <code>note · description · dialogue · preset</code>                                                                                                   |
| PROMPT    | <code>prompt · preset</code>                                                                                                                          |
| EMPTY     | <code>internal_lib_empty · template_slot</code>                                                                                                       |

Mutabilité du profile

**Le profile d'un Block non-container peut changer.** C'est une opération métier normale — par exemple promouvoir une image générée en référence validée :

```
# Une image générée par IA, validée et promue en référence de personnage
block.profile = "reference" # était "generated" — autorisé car type=IMAGE
```

**Le profile d'un Block CONTAINER est fixe.** Le rôle structurel d'un container (`shot`, `character`, `workspace_root`...) est déterminé à sa création et ne change pas — le modifier briserait la hiérarchie et les règles de validation de `ContainerRulesService`.

## InputConnection — Liens typés entre Blocks

Défini dans `src/domain/models.py`. Un Block cible expose ses connexions entrantes via `block.inputs`.

```
@dataclass(slots=True)
class InputConnection:
    source_block_id: str # Block amont (source du lien) — toujours un Block non-container
    port: PortType # Port logique sur le Block cible
    name: str = "" # Label visible (ex: "start_frame", "voice_ref")
    enabled: bool = True
    order: int = 0 # Ordre déterministe pour l'affichage
    metadata: dict[str, Any] # Payload arbitraire par connexion
```

**Contrainte actuelle :** les connexions se font **uniquement entre Blocks unitaires** (IMAGE, VIDEO, AUDIO, TEXT, PROMPT) — jamais entre Blocks **CONTAINER**. Les containers organisent et contiennent ; ils ne sont pas connectés entre eux.

**Principe de lecture :**

Pour savoir ce qui est connecté à un Block, lire `block.inputs`.

Pour trouver tous les Blocks qui *utilisent* un Block source, scanner `inputs` de tous les Blocks.

### ⚠ Dette de refactoring — connexions entre containers

La connexion entre containers (ex: relier un Shot à un personnage via son container) n'est pas implémentée. C'est une évolution future qui nécessitera de définir des règles de validation dans `ContainerRulesService` pour déterminer quels types de containers peuvent se connecter et sur quels ports.

## Couches d'édition structurelles — FreeTree et FreeGraph

Les Blocks **CONTAINER** embarquent deux couches d'édition UI **indépendantes**, activées à la demande par l'utilisateur.

- **FreeTree** est une couche d'**architecture** : elle permet à l'utilisateur d'organiser les Blocks en hiérarchie avec des dossiers virtuels, indépendamment de la liste brute **block.contains**. L'arbre peut ne refléter qu'une sous-sélection ou un regroupement thématique des enfants.
- **FreeGraph** est une couche de **liens** : elle permet à l'utilisateur de visualiser et d'éditer spatialement les connexions entre Blocks, indépendamment de l'architecture des containers. Les nœuds sont positionnés librement ; les arêtes représentent des **InputConnection** ou des relations visuelles.

Ces deux couches sont **orthogonales à block.contains** : **contains** reste la source de vérité sur l'appartenance ; **FreeTree** et **FreeGraph** en sont des lectures enrichies et éditables par l'utilisateur.

Les services **FreeTreeService** et **FreeGraphService** initialisent et maintiennent ces couches à la demande — ne jamais instancier **FreeTree()** / **FreeGraph()** directement.

FreeTree — couche architecture

```
@dataclass(slots=True)
class FreeTree:
    root_ids: list[str]          # IDs des nœuds racine (ordre)
    nodes: dict[str, FreeTreeNode] # Nœuds indexés par node_id

    @dataclass(slots=True)
    class FreeTreeNode:
        id: str                  # ID local à l'arbre (ex: "node_xxx")
        kind: str                 # "folder" | "block_ref"
        name: str
        block_id: str | None = None # Référence vers un Block (si block_ref)
        children: list[str]       # IDs enfants (node_ids)
```

Un **folder** est un nœud virtuel sans Block associé.  
Un **block\_ref** est un pointeur nommé vers un Block existant (**block\_id**).

FreeGraph — Graphe spatial positionné

```
@dataclass(slots=True)
class FreeGraph:
    nodes: dict[str, FreeGraphNode]
    edges: dict[str, FreeGraphEdge]

    @dataclass(slots=True)
    class FreeGraphNode:
        id: str                  # ID local au graphe ("gnode_xxx")
        block_id: str             # Block représenté
        x: float = 0.0
        y: float = 0.0

    @dataclass(slots=True)
    class FreeGraphEdge:
        id: str                  # ID local ("gedge_xxx")
        source_node_id: str      # gnode_xxx source
        target_node_id: str      # gnode_xxx cible
        label: str = ""
```

Règles de synchronisation — **contains** / **tree** / **graph**

Les trois structures ne se synchronisent **pas automatiquement**. Chacune a un rôle distinct :

| Structure             | Rôle                                    | Source de vérité                  |
|-----------------------|-----------------------------------------|-----------------------------------|
| <b>block.contains</b> | Liste canonique des IDs enfants         | ✅ <b>Oui — c'est la référence</b> |
| <b>block.tree</b>     | Vue hiérarchique avec dossiers virtuels | Non — projection optionnelle      |
| <b>block.graph</b>    | Vue spatiale positionnée                | Non — projection optionnelle      |

## Règle 1 — **contains** est la source de vérité

Un Block est enfant d'un container si et seulement si son **id** figure dans **container.contains**.

**FreeTree** et **FreeGraph** sont des **vues décoratives** sur **contains** — elles ne définissent pas l'appartenance.

## Règle 2 — **FreeGraph** enforce l'appartenance à **contains**

**FreeGraphService.add\_node()** vérifie que **block\_id in container.contains** avant d'ajouter un nœud au graphe. Violation → **ValidationError**.

```
# Enforced dans FreeGraphService.add_node()
if block_id not in container.contains:
    raise ValidationError("Block must be inside container before graph placement")
```

## Règle 3 — **FreeTree** n'enforce pas l'appartenance

**FreeTreeService** ne vérifie pas que le **block\_id** d'un **block\_ref** est dans **container.contains**.

Un nœud orphelin (pointant vers un Block supprimé ou déplacé) peut exister silencieusement.

C'est l'appelant (**UseCaseService**) qui est responsable du nettoyage.

## Règle 4 — **Suppression : cascade manuelle obligatoire**

Quand un Block est retiré de **container.contains**, rien n'est nettoyé automatiquement.

L'appelant doit :

1. Retirer le **block\_ref** correspondant dans **block.tree** (via **FreeTreeService.remove\_node()**)
2. Retirer le nœud et ses arêtes dans **block.graph** (via **FreeGraphService.remove\_node()**)
3. Mettre à jour **container\_paths** sur le Block retiré

## Règle 5 — **container\_paths** suit **tree**

**block.container\_paths** est un dict **{container\_id → chemin virtuel dans FreeTree}**.

Il est maintenu par **FreeTreeService** lors des déplacements de nœuds.

Si un nœud **block\_ref** est supprimé de l'arbre, la clé correspondante doit être retirée de **container\_paths** du Block pointé.

## Résumé — qui fait quoi

```
Ajouter un Block dans un container :
UseCaseService
├─ BlockService.add_to_container() → met à jour container.contains
├─ FreeTreeService.add_node() → ajoute un block_ref dans tree (optionnel)
└─ FreeGraphService.add_node() → ajoute un node dans graph (optionnel)
    ▲ exige que block_id ∈ contains

Supprimer un Block d'un container :
UseCaseService
├─ FreeTreeService.remove_node() → retire le block_ref + met à jour container_paths
├─ FreeGraphService.remove_node() → retire le node + toutes ses arêtes
└─ BlockService.remove_from_container() → retire l'id de contains (en dernier)
```

## Contenu typé — **MediaContent** et **ContainerContent** ✓

Défini dans **src/domain/block\_content.py**.

**block.content** est un **dict[str, Any]** libre utilisé comme format de persistance.

**Toujours lire via les accesseurs typés** — ne jamais écrire de code qui appelle **block.content.get("...")** directement dans les services et l'UI.

Lecture (accesseurs snapshot)

```
# Block IMAGE / VIDEO / AUDIO
media = block.as_media()          # → MediaContent
media.storage_path                # str – chemin fichier principal
media.thumbnail_path              # str – vignette
media.preview_path                # str – preview léger (vidéo)
media.width                       # int – px (0 = inconnu)
```

```
media.height                # int - px
media.duration_ms           # int - durée ms (0 = N/A)
media.candidate_paths()     # tuple[str, ...] - chemins non-vides par priorité

# Block CONTAINER / workspace_root
ctn = block.as_container()   # → ContainerContent
ctn.workspace_role           # str - "story_root", "characters_root"... (lowercase)
ctn.internal_lib             # bool
ctn.drop_target              # bool
```

Écriture (directement dans le dict backing)

```
# Écrire avec les clés canoniques - le dict est persisté tel quel en JSON
block.content["storage_path"] = str(destination)
block.content["thumbnail_path"] = str(thumb)
block.content["workspace_role"] = "story_root"
block.content["internal_lib"] = True
```

#### ⚠ Dette de refactoring — asymétrie lecture / écriture

La lecture passe par des accesseurs typés (`as_media()`, `as_container()`) mais l'écriture reste directe dans le dict brut. Cette asymétrie tient à une raison technique valide : `block_to_dict()` dans `serialization.py` sérialise `block.content` tel quel — un accesseur d'écriture devrait donc aussi écrire dans le dict pour que la persistance fonctionne sans modification.

La cible à terme est d'ajouter des accesseurs d'écriture symétriques dans `block_content.py` qui écrivent *dans* le dict backing :

```
# Cible future (non implémenté)
block.set_media(storage_path="storage/files/hulk.png", width=1920, height=1080)
block.set_container(workspace_role="story_root")
```

En attendant : **toujours utiliser les clés canoniques** documentées dans `block_content.py` pour les écritures — ne jamais inventer de nouvelles clés sans les y documenter.

🔧 PromptContent (planifié)

```
# Futur accesseur pour block.as_prompt() → PromptContent
# Champs prévus :
#   text: str          - texte du prompt final
#   negative: str       - negative prompt
#   style_tags: list[str]
#   target_service: str - "openart" | "veo" | "gemini" | "elevenlabs"
#   model_id: str
#   seed: int
```

## Provenance — Traçabilité d'origine

`block.provenance` est un dict stocké tel quel en JSON.

La clé `"kind"` (valeur de `BlockProvenanceKind`) est toujours présente.

Provenance locale

```
provenance = {"kind": "local"}
```

Provenance Clone (LIB\_CLONE)

```
provenance = {
    "kind": "lib_clone",
    "mount_id": "lib_mount_ff1a131b",
    "source_workspace_id": "library_a1b2c3d4",
}
```

```
"source_workspace_path": "/Users/.../BASE.sbcprj",
"source_block_id": "blk_image_XXXXXXXXXX",
"source_block_name": "Hulk Rage — face",
}
```

Provenance Link (LIB\_LINK)

Mêmes champs que LIB\_CLONE + `access_mode = LINK`.  
Le Block est en lecture seule ; sa mise à jour depuis la source est 🗑️ planifiée.

🔧 Assets générés par IA — provenance et paramètres (planifié)

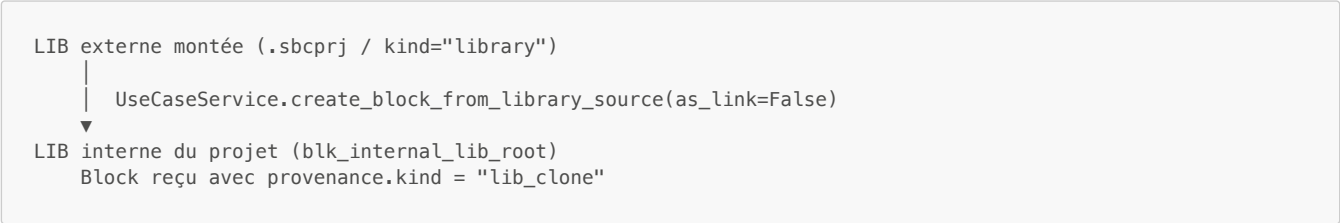
Les assets générés (IMAGE, VIDEO, AUDIO) gardent `provenance.kind = "local"`.  
Les informations de génération sont portées par `block.content` — voir la section  
"Enrichissement futur — assets générés par IA" dans les schémas de Blocks.

Système Clone / Link — Flux en deux étapes ✓

L'import d'un Block depuis une librairie externe suit un **flux obligatoire en deux étapes**.  
La LIB interne est le point de passage intermédiaire — jamais d'import direct vers un workspace domaine.

Étape 1 — LIB externe montée → LIB interne du projet

La première étape ramène le Block de la librairie externe vers la LIB interne  
(`blk_internal_lib_root`) du projet courant.

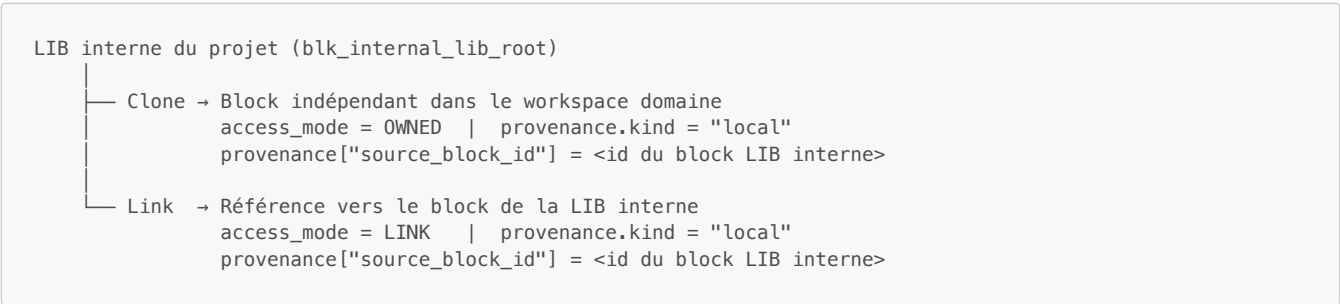


Le Block ainsi importé appartient au projet — c'est **une instance propre au projet**,  
indépendante de la librairie source (sauf si `as_link=True`, auquel cas il reste  
lecture seule et lié à la source).

| Opération               | access_mode | provenance.kind | Éditable | Suit les mises à jour source |
|-------------------------|-------------|-----------------|----------|------------------------------|
| Clone LIB → LIB interne | OWNED       | lib_clone       | ✓ Oui    | ✗ Non (copie indépendante)   |
| Link LIB → LIB interne  | LINK        | lib_link        | ✗ Non    | 🗑️ Oui (planifié)            |

Étape 2 — LIB interne → workspace domaine du projet

Une fois le Block dans la LIB interne, l'utilisateur peut l'utiliser dans ses  
workspaces domaines (Personnages, Story...) de deux façons :



| Opération                   | access_mode | provenance.kind | Éditable | Intérêt                                                       |
|-----------------------------|-------------|-----------------|----------|---------------------------------------------------------------|
| Clone LIB interne → domaine | OWNED       | local           | ✓ Oui    | Copie libre, peut diverger du block LIB interne               |
| Link LIB interne → domaine  | LINK        | local           | ✗ Non    | Pointe vers le block LIB interne — 1 seul endroit à maintenir |

**Note provenance :** les opérations depuis la LIB interne utilisent `"local"`  
(pas `"lib_clone"` / `"lib_link"`) car la source est dans le même projet.  
Le champ `provenance["source_block_id"]` trace l'origine sans nécessiter  
de nouvelle valeur dans l'enum `BlockProvenanceKind`.



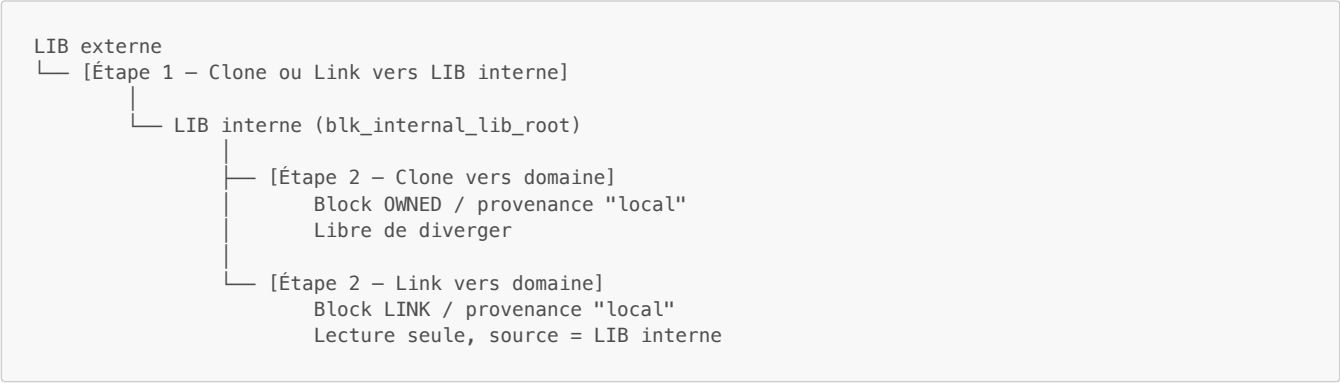


Tableau de synthèse — toutes les origines possibles

| Origine                  | access_mode | provenance.kind         | Éditable |
|--------------------------|-------------|-------------------------|----------|
| Créé localement          | OWNED       | local                   | ✔ Oui    |
| Clone depuis LIB externe | OWNED       | lib_clone               | ✔ Oui    |
| Link depuis LIB externe  | LINK        | lib_link                | ✗ Non    |
| Clone depuis LIB interne | OWNED       | local + source_block_id | ✔ Oui    |
| Link depuis LIB interne  | LINK        | local + source_block_id | ✗ Non    |

Lien cassé → 🗑️ Proposer : reconvertir en Clone via snapshot, ou supprimer.

Badge UI : [Link] orange · [Clone src=LIB] vert · [Clone src=interne] vert clair · [Local] gris

## Projet et Librairie — Workspaces filesystem ✔

Un **projet** et une **librairie** sont des **workspaces filesystem** — des répertoires `.sbcprj` sur le disque. Ils partagent exactement le même format ; seul le champ `kind` dans `project.json` les distingue ("`project`" ou "`library`").

### Racines Master — le concept fondamental ✔

À l'intérieur d'un workspace, les Blocks sont organisés autour de **Racines Master** (`profile="workspace_root"`). Une Racine Master est un Block **CONTAINER** qui chapeaute un ensemble logique de Blocks. Chaque Racine Master génère un répertoire de partition dans `workspaces/` et son propre `blocks.json`.



Un projet a typiquement deux Racines Master :

| Racine Master      | ID                    | Rôle                                                                       |
|--------------------|-----------------------|----------------------------------------------------------------------------|
| Racine Projet      | blk_project_root      | Chapeau principal — contient les racines domaines                          |
| Racine InternalLib | blk_internal_lib_root | Espace de travail local pour les Blocks issus des LIBs ou créés localement |

La Racine Projet (`blk_project_root`) est elle-même container des racines domaines :



Rôle de la LIB interne (`blk_internal_lib_root`) ✔

La LIB interne est l'espace de travail **local au projet** destiné à accueillir deux types de Blocks :

1. **Clones issus de LIBs montées** — quand l'utilisateur importe un Block depuis une librairie externe (Clone ou Link), ce Block atterrit dans la LIB interne avant d'être éventuellement déplacé dans un workspace domaine (Personnages, Story...). C'est la zone de réception et d'organisation des ressources venues de l'extérieur.
2. **Blocks créés directement** — l'utilisateur peut créer des Blocks containers directement dans la LIB interne pour constituer une bibliothèque locale au projet (assets maison, presets locaux, références visuelles propres au film).

**Ce que la LIB interne n'est pas** : ce n'est pas une librairie partageable.

Les Blocks qu'elle contient restent privés au projet. Pour partager des Blocks vers d'autres projets, il faut les promouvoir dans une LIB externe (`.sbcprj` de type `"library"`) montée ailleurs.

**Drop zone** : le Block `blk_internal_lib_empty` (`profile="internal_lib_empty", type=EMPTY`) sert de cible visuelle pour le glisser-déposer — il signale à l'utilisateur où déposer un asset pour l'importer dans la LIB interne.

**Une librairie peut avoir plusieurs Racines Master**, chacune représentant un projet-source indépendant (templates de personnages, décors, ressources audio...). Chaque Racine Master de la librairie est accessible indépendamment et peut être montée ou clonée dans un projet cible.

Organisation filesystem 

```
<nom>.sbcprj/
├── project.json           ← Métadonnées du workspace (kind, mounted_libraries...)
├── ui_state.json         ← État UI (sélection, scroll, panneaux ouverts)
├── storage/
│   ├── files/            ← Médias importés (PNG, MP4, MP3...)
│   └── thumbs/           ← Vignettes pré-générées
├── cache/
│   └── previews/         ← Previews vidéo extraites (JPEG)
└── workspaces/           ← 1 répertoire par Racine Master
    ├── project_root/     ← Racine Projet + racines domaines orphelines
    │   └── blocks.json
    ├── characters_root/  ← Blocks du domaine Personnages
    │   └── blocks.json
    ├── story_root/       ← Blocks du domaine Scénario
    │   └── blocks.json
    ├── library_root/     ← Blocks du domaine Librairie externe
    │   └── blocks.json
    └── internal_lib/     ← LIB interne (clones de LIBs montées + créations locales)
        └── blocks.json
```

**Règle de partitionnement** : chaque Block est rangé dans le `blocks.json` de la Racine Master qui le contient (directement ou via des containers enfants). Les Blocks non assignés tombent dans le partition de `blk_project_root` (fallback).

project.json

```
{
  "id": "project_a1b2c3d4",
  "kind": "project",
  "name": "Mon Film",
  "version": 1,
  "storage_layout_version": 2,
  "description": "",
  "preview_image_path": "storage/files/cover.png",
  "author_name": "",
  "author_email": "",
  "mounted_libraries": [
    {
      "id": "lib_mount_ff1a131b",
      "kind": "LIB",
      "path": "/Users/alice/Librairies/BASE.sbcprj",
      "label": "BASE",
      "enabled": true,
    }
  ]
}
```

```

    "read_only": true,
    "mounted_at": "2026-04-08T14:09:49Z"
  }
],
"created_at": "2026-04-01T15:00:15Z",
"updated_at": "2026-04-17T10:00:00Z"
}

```

blocks.json (un par workspace)

```

[
  {
    "id": "blk_image_a3f7c9d1e2b0",
    "type": "image",
    "profile": "generated",
    "name": "Hulk Rage – plan large",
    "description": "Plan large forêt, crépuscule",
    "prompt_ref": "Wide shot, Hulk emerging from dark forest, golden hour",
    "prompt_generated": "Wide shot, Hulk emerging from dark forest, golden hour, cinematic, 8K",
    "comment": "",
    "shared": false, // Δ toujours false – champ obsolète
    "domain": "characters",
    "access_mode": "owned",
    "provenance": {
      "kind": "lib_clone",
      "mount_id": "lib_mount_ff1a131b",
      "source_block_id": "blk_image_xxxxxxxxxxxx",
      "source_block_name": "Hulk Rage ref"
    },
    "functional_name": "",
    "tags": ["hulk", "forest", "wide"],
    "content": {
      "storage_path": "storage/files/hulk_rage_wide.png",
      "thumbnail_path": "storage/thumbs/hulk_rage_wide.jpg",
      "width": 1920,
      "height": 1080
    },
    "contains": [],
    "inputs": [],
    "container_paths": {},
    "tree": null,
    "graph": null
  }
]

```

IDs des Blocks racine (hardcodés dans main\_window.py)

```

# Racines Master (profile="workspace_root", type=CONTAINER)
PROJECT_ROOT_BLOCK_ID = "blk_project_root" # Racine Master principale
INTERNAL_LIB_ROOT_BLOCK_ID = "blk_internal_lib_root" # Racine Master librairie interne

# Racines domaine (enfants de blk_project_root)
CHARACTERS_ROOT_BLOCK_ID = "blk_characters_root"
STORY_ROOT_BLOCK_ID = "blk_story_root"
LIB_ROOT_BLOCK_ID = "blk_lib_root"

# Block technique (enfant de blk_internal_lib_root)
INTERNAL_LIB_EMPTY_BLOCK_ID = "blk_internal_lib_empty" # Drop zone EMPTY

```

Toutes les racines sont des Blocks `CONTAINER` / `profile="workspace_root"`.

Leur rôle sémantique est lu via `block.as_container().workspace_role` (ex: "project\_root", "story\_root", "internal\_lib").

## Architecture en couches

```

src/
├── domain/                # Entités stables – Block, enums, block_content
│   ├── models.py         # Block, FreeTree, FreeGraph, InputConnection, tous les enums
│   └── block_content.py   # MediaContent, ContainerContent (typed content views)

```

|  |                       |                                                             |
|--|-----------------------|-------------------------------------------------------------|
|  | exceptions.py         | # DomainError, NotFoundError, ValidationError               |
|  | infrastructure/       |                                                             |
|  | repositories/         | # BlockRepository (dict en mémoire – extensible)            |
|  | storage/              | # WorkspaceStorageService, serialization, csv_seed          |
|  | services/             | # CRUD bas niveau                                           |
|  | block_service.py      | # BlockService – CRUD + ops container + InputConnection     |
|  | free_tree_service.py  | # FreeTreeService – arbre avec dossiers                     |
|  | free_graph_service.py | # FreeGraphService – graphe spatial                         |
|  | container_rules.py    | # ContainerRulesService – validation hiérarchie             |
|  | application/          | # Orchestration haut niveau                                 |
|  | use_case_service.py   | # UseCaseService – façade unique pour l'UI                  |
|  | container_resolver.py | # ContainerResolver – projection UI d'un container          |
|  | workspaces/           | # ProjectWorkspaceService, CharacterWorkspaceService...     |
|  | UI/                   |                                                             |
|  | windows/              | # MainWindow, dialogs, fenêtres secondaires                 |
|  | Frames/workspaces/    | # Panneaux par workspace (Project, Characters, Story...)    |
|  | Widgets/              | # Composants réutilisables (grid, tree, graph, carousel...) |
|  | themes/               | # Tokens de design, QSS, ThemeLoader                        |

### UseCaseService — façade principale

Point d'entrée unique pour l'UI.

La règle "*ne pas appeler **BlockService** directement depuis **UI***" est une convention architecturale — **elle doit être vérifiée mécaniquement** pour survivre aux refactorings.

Voir la section "**Enforcement de l'architecture en couches**" ci-dessous.

```
# Créer un Block
use_case.create_block(type=BlockType.IMAGE, domain=BlockDomain.CHARACTERS,
                      profile="generated", name="Hulk Rage", ...)

# Créer un Block dans un container
use_case.create_block_in_container(parent_container_id="blk_characters_root", ...)

# Clone / Link depuis librairie
use_case.create_block_from_library_source(
    source_block=lib_block,
    mount_id="lib_mount_ff1a131b",
    source_workspace_id="library_a1b2",
    source_workspace_path="/path/to/lib.sbcprj",
    as_link=False,           # False = clone, True = link
    parent_container_id="blk_characters_root",
)

# Connecter deux Blocks unitaires (InputConnection)
# Source et cible sont des Blocks non-container (IMAGE, VIDEO, AUDIO, TEXT, PROMPT)
use_case.connect_input(
    target_block_id="blk_image_yyyy",    # Block cible – reçoit la connexion
    source_block_id="blk_prompt_xxxx",   # Block source – fournit le lien
    port=PortType.IN,
    name="prompt_source",
)
```

## Enforcement de l'architecture en couches

### Problème

La règle "*l'UI n'appelle jamais **BlockService** directement*" est aujourd'hui une **convention documentée mais non vérifiée** : Python ne l'interdit pas, il n'y a pas de tests d'architecture, et aucun linter ne la détecte. Une telle règle disparaît silencieusement dès la deuxième session de refactoring.

### Solution — **import-linter**

**import-linter** est un outil dédié à l'enforcement des frontières architecturales via des contrats déclarés dans

pyproject.toml. Il s'exécute en CI ou en pre-commit hook et échoue si un import interdit est détecté.

Installation

```
pip install import-linter
```

Configuration à ajouter dans pyproject.toml

```
[tool.importlinter]
root_packages = ["domain", "infrastructure", "services", "application", "UI"]

[[tool.importlinter.contracts]]
name = "UI ne peut pas importer les services directement"
type = "forbidden"
source_modules = ["UI"]
forbidden_modules = [
    "services.block_service",
    "services.free_tree_service",
    "services.free_graph_service",
    "services.container_rules",
]

[[tool.importlinter.contracts]]
name = "domain ne dépend de rien d'autre"
type = "independence"
modules = ["domain"]

[[tool.importlinter.contracts]]
name = "services ne peuvent pas importer UI"
type = "forbidden"
source_modules = ["services", "application"]
forbidden_modules = ["UI"]
```

Exécution

```
lint-imports          # vérifie tous les contrats
lint-imports --debug  # détail des imports analysés
```

Intégration pre-commit (.pre-commit-config.yaml à créer)

```
repos:
- repo: local
  hooks:
  - id: architecture-contracts
    name: "Architecture – vérification des frontières de couches"
    language: system
    entry: lint-imports
    pass_filenames: false
    always_run: true
```

Règles d'architecture vérifiables

| Règle                                   | Type de contrat    | Vérifié par   |
|-----------------------------------------|--------------------|---------------|
| UI n'importe pas services.* directement | forbidden          | import-linter |
| domain n'a aucune dépendance externe    | independence       | import-linter |
| services n'importe pas UI               | forbidden          | import-linter |
| Pas de block.content.get() dans UI/     | grep / pygrep hook | pre-commit    |
| Pas de BlockService dans UI/            | forbidden          | import-linter |

Hook complémentaire — contenu typé (.pre-commit-config.yaml)

```
- repo: local
hooks:
  - id: no-raw-content-access
    name: "Pas de block.content.get() dans UI/ et application/"
    language: pygrep
    entry: 'block\.content\.get\('
    files: ^src/(UI|application)/
    types: [python]
```

Statut actuel

| Contrat                  | Statut                         |
|--------------------------|--------------------------------|
| import-linter installé   | ❌ Non                          |
| pyproject.toml configuré | ❌ Non                          |
| pre-commit configuré     | ❌ Non                          |
| Contrats écrits          | 🔧 Spécifiés ici, à implémenter |

Workspaces — Navigation UI ✔

[ Dashboard ] | [ Projet ] [ Personnages ] [ Scénario ] [ Librairies ] [ Paramètres ]

| Clé nav            | Page dans le stack                                         | Contenu réel                        | Workspace root      |
|--------------------|------------------------------------------------------------|-------------------------------------|---------------------|
| "dashboard"        | _workspace_dashboard_page (QWidget inline dans MainWindow) | ProjectWorkspacePanel + stats tiles | blk_project_root    |
| "asset_library"    | _workspace_asset_library_page                              | LibraryWorkspacePanel               | blk_lib_root        |
| "character_studio" | _workspace_character_studio_page                           | CharacterWorkspacePanel             | blk_characters_root |
| "story"            | _workspace_story_page                                      | StoryWorkspacePanel                 | blk_story_root      |
| "settings"         | _workspace_settings_page                                   | SettingsWorkspacePanel              | —                   |
| "support"          | _workspace_support_page                                    | EmptyStateWidget                    | —                   |

⚠ Dette de refactoring — Dashboard à extraire

Le Dashboard (`_workspace_dashboard_page`) est actuellement un `QWidget` construit inline dans `MainWindow`, avec ses stats tiles (`_dashboard_stats_frame`, `_dashboard_stat_tiles`, `_refresh_dashboard_stats()`) directement dans le code de la fenêtre principale.

Lors d'un prochain refactoring, il devra être extrait en sa propre classe `DashboardWorkspacePanel` dans `UI/Frames/workspaces/`, sur le même modèle que `ProjectWorkspacePanel`, `CharacterWorkspacePanel`, etc. La logique de stats (`_build_dashboard_stat_tiles`, `_refresh_dashboard_stats`, `_project_stats_view`) doit migrer dans cette classe, et `MainWindow` ne conserve que le câblage.

Fenêtres secondaires ✔

| Fenêtre                   | Description                      |
|---------------------------|----------------------------------|
| ThumbnailListWindow       | Navigateur d'assets en vignettes |
| MediaCarouselWindow       | Visionneur plein écran           |
| FreeTreeWindow            | Éditeur d'arbre dédié            |
| ProjectVisualPickerDialog | Sélecteur d'image modal          |

Cycle de vie d'un projet — Session ✔

Séquence de chargement



Qui instancie quoi

| Objet                                                       | Instancié par                                         | Quand                                           |
|-------------------------------------------------------------|-------------------------------------------------------|-------------------------------------------------|
| BlockRepository                                             | Fonction de chargement/mise à jour dans<br>MainWindow | À chaque <b>Open Project</b> ou<br>rechargement |
| WorkspaceStorageService                                     | MainWindow.__init__                                   | Démarrage de l'application                      |
| ProjectWorkspaceService,<br>CharacterWorkspaceService, etc. | MainWindow.__init__                                   | Démarrage de l'application                      |
| UseCaseService                                              | En fonction du block en cours de traitement           | Par action utilisateur — contexte par<br>block  |
| Panels UI (CharacterWorkspacePanel, etc.)                   | MainWindow.__init__                                   | Démarrage de l'application                      |

Chargement automatique au démarrage

MainWindow.\_\_init\_\_ essaie dans l'ordre :

- 1. Blocks passés en paramètre (test / deep link)
- 2. \_load\_blocks\_safely() sur le chemin courant
- 3. \_user\_config.load\_last\_project\_path() — dernier projet ouvert

Partage des Blocks entre panels

self.\_blocks: list[Block] dans MainWindow est l'état central en mémoire.

Chaque panel reçoit la liste complète via panel.set\_blocks(self.\_blocks).  
Il sélectionne lui-même les Blocks à afficher en filtrant sur :

- block.domain (ex: BlockDomain.CHARACTERS)
- la descendance d'un workspace\_root donné (ex: blk\_characters\_root)
- block.profile pour des vues spécialisées

Les panels **ne modifient jamais self.\_blocks directement** — toute mutation passe par UseCaseService qui notifie MainWindow de rafraîchir la liste et de rappeler set\_blocks() sur tous les panels concernés.

Schémas de Blocks courants

Block IMAGE / VIDEO / AUDIO 

```
Block(
    id="blk_image_a3f7c9d1e2b0",
    type=BlockType.IMAGE,
    profile="generated",      # ou "reference", "asset", "variation"
    name="Hulk Rage — plan large",
    domain=BlockDomain.CHARACTERS,
    prompt_ref="Wide shot, Hulk...",
    prompt_generated="Wide shot, Hulk..., cinematic 8K",
```

```

tags=["hulk", "forest"],
content={
  "storage_path": "storage/files/hulk.png",
  "thumbnail_path": "storage/thumbs/hulk.jpg",
  "width": 1920,
  "height": 1080,
  # VIDEO uniquement :
  "preview_path": "cache/previews/hulk_preview.jpg",
  "duration_ms": 4000,
}
)

```

### 🔧 Enrichissement futur — assets générés par IA

Quand un asset (IMAGE, VIDEO, AUDIO) est produit par un service IA externe, des informations de génération devront être persistées. Elles iront dans `block.content` avec les clés canoniques suivantes (à documenter dans `block_content.py` lors de l'implémentation) :

```

content={
  # ... champs media existants (storage_path, width, etc.) ...

  # Identification du service générateur
  "ai_service": "openart",          # "openart" | "veo" | "gemini" | "elevenlabs" | "suno"
  "ai_model": "flux-1.1-pro",      # identifiant exact du modèle utilisé

  # Paramètres de génération (reproductibilité)
  "ai_seed": 42,                  # seed pour régénérer à l'identique
  "ai_params": {                  # paramètres libres propres au service
    "steps": 30,
    "style": "cinematic",
    "aspect_ratio": "16:9",
    # audio : "voice_id", "stability", "similarity_boost"...
    # vidéo : "duration_sec", "camera_motion"...
  },

  # Traçabilité et coût
  "ai_generated_at": "2026-04-17T10:00:00Z",
  "ai_cost_usd": 0.04,           # coût unitaire de cette génération
  "ai_prompt_block_id": "blk_prompt_xxxx", # Block PROMPT source utilisé
}

```

La `provenance` de ces Blocks reste `"local"` (valeur existante dans l'enum) — l'origine IA est portée par les clés `content["ai_service"]` et non par un nouveau `BlockProvenanceKind`. Un futur accesseur `block.as_generated()` pourra exposer ces champs de façon typée (analogue à `as_media()` et `as_container()`).

Pour l'agrégation des coûts : scanner tous les Blocks dont `content.get("ai_service")` est renseigné et sommer `content.get("ai_cost_usd", 0.0)`.

### Block PROMPT ✅

```

Block(
  id="blk_prompt_xxxxxxxxxxxx",
  type=BlockType.PROMPT,
  profile="prompt",
  name="Shot 01 – Brief visuel",
  domain=BlockDomain.STORY,
  prompt_ref="Wide shot, Hulk emerges from dark forest at golden hour",
  prompt_generated="Wide shot, Hulk emerges from dark forest at golden hour, "
    "cinematic photography, 8K, dramatic lighting, RAW",
  tags=["shot01", "forest", "hulk"],
  # 🔧 futur : content["target_service"] = "openart"
)

```

### Block CONTAINER — Shot ✅



```

Block(
    id="blk_container_xxxxxxxxxx",
    type=BlockType.CONTAINER,
    profile="shot",
    name="Shot 01 – Arrivée forêt",
    domain=BlockDomain.STORY,
    description="Plan large, Hulk émerge de la forêt, lumière dorée",
    contains=[
        "blk_prompt_aaaa", # Brief visuel
        "blk_image_bbbb", # Image sélectionnée
        "blk_video_cccc", # Vidéo générée (Veo)
        "blk_audio_dddd", # Voix off (ElevenLabs)
    ],
    inputs=[], # Les containers n'ont pas de connexions à ce stade
    # tree et graph sont initialisés par les services (FreeTreeService, FreeGraphService)
    # au fur et à mesure que l'utilisateur édite la couche arbre ou la couche liens.
    # Ne pas instancier FreeTree() / FreeGraph() manuellement dans le code applicatif.
)

# Les liens se font entre Blocks unitaires à l'intérieur du container.
# Exemple : le Block IMAGE peut recevoir une connexion d'un Block PROMPT source.
Block(
    id="blk_image_bbbb",
    type=BlockType.IMAGE,
    profile="generated",
    name="Hulk Rage – plan large",
    inputs=[
        InputConnection(source_block_id="blk_prompt_aaaa",
                        port=PortType.IN, name="prompt_source", order=0),
    ],
)

```

## Connexion aux outils IA — Évolution future

**Cette section décrit une évolution future, rien n'est implémenté.**

Le modèle de données Block est conçu pour l'accueillir sans modification

structurelle — les champs `prompt_ref`, `prompt_generated`, `content["ai_service"]`, `content["ai_params"]` sont prévus à cet effet.

### Intention

Connecter SBC à des services IA externes (OpenArt, Veo, Gemini, ElevenLabs...) via un module bridge dédié, pour générer des Blocks (IMAGE, VIDEO, AUDIO) depuis un Block PROMPT sans quitter l'application.

### Services à créer (non implémentés)

```

services/ai_bridge/    ← à créer
├── base_provider.py    # Interface commune
├── openart_provider.py # Prompt → Block IMAGE
├── veo_provider.py     # Prompt + image → Block VIDEO
├── elevenlabs_provider.py # Text → Block AUDIO
├── gemini_provider.py  # Enrichissement de prompt
└── job_queue_service.py # File d'attente et statut

```

### Flux envisagé

```

Block PROMPT → [service IA] → Block IMAGE / VIDEO / AUDIO
                                content["ai_service"] = "openart"
                                content["ai_model"]    = "flux-1.1-pro"
                                content["ai_seed"]     = 42
                                provenance.kind       = "local"

```

### Profil à ajouter lors de l'intégration (non implémentés)

```
"shot_brief" . "generation_job" . "voice_line" . "storyboard_frame" . "scene"
```

## Signaux Qt — Conventions et catalogue

### Principe général — flux à deux sens

Les modifications de Blocks suivent un flux **strictement asymétrique** :

```
— BOTTOM-UP : intention de mutation —  
Widget → Signal → Panel → Signal → MainWindow._handler()  
                                     |  
                               UseCaseService  
                               self._blocks mis à jour
```

```
— TOP-DOWN : propagation de l'état —  
MainWindow._refresh_project_workspace()  
→ panel.set_blocks(self._blocks, ...) ← TOUS les panels  
→ chaque panel filtre et se redessine
```

#### Règle fondamentale :

Toute mutation de Block passe par `UseCaseService` dans `MainWindow`.

Après mutation, `MainWindow` appelle `_refresh_project_workspace()` qui pousse `self._blocks` vers **tous** les panels et fenêtres secondaires.

**Aucun composant ne garde une copie locale de Blocks — il travaille sur ce que `set_blocks()` lui a fourni.**

Règle de propagation — composants affichant des données de Block

**Tout composant qui affiche des données d'un Block (nom, image, propriété, position dans un graphe, chemin dans un arbre... ) DOIT se rafraîchir intégralement dès que `set_blocks()` est appelé sur lui.**

Conséquences pour l'implémentation :

```
#  Correct — le panel stocke la référence et redessine sur set_blocks()  
class MonPanel(QWidget):  
    def set_blocks(self, blocks: list[Block], project_root: str = "") -> None:  
        self._blocks = blocks                # remplace — jamais de merge partiel  
        self._project_root = project_root  
        self._refresh_view()                 # redessine tout depuis _blocks  
  
#  Interdit — copie partielle, l'état diverge de MainWindow  
class MonPanel(QWidget):  
    def update_one_block(self, block: Block) -> None:  
        self._cache[block.id] = block        # la liste _blocks est désynchronisée
```

Flux complet d'une modification de Block

```
Utilisateur édite un champ dans BlockPropertiesEditor  
|  
▼  
block_properties_editor.property_change_requested.emit({"block_id": "...", "name": "..."})  
  (dict avec les champs modifiés)  
|  
▼  
Panel (CharacterWorkspacePanel / StoryWorkspacePanel)  
  block_update_requested.emit(payload)      ← relaie tel quel  
|  
▼  
MainWindow._update_character_block_from_workspace(payload)  
  use_case.update_block(...)                ← mutation sur self._blocks via UseCaseService  
  self._persist_project_blocks(self._blocks) ← sauvegarde JSON  
  self._refresh_project_workspace()         ← redistribue à tous les panels  
  |  
  ├── _character_workspace_panel.set_blocks(self._blocks, ...)  
  ├── _story_workspace_panel.set_blocks(self._blocks, ...)  
  ├── _thumbnail_window.set_blocks(self._blocks, ...) (si ouverte)  
  ├── _media_carousel_window.set_blocks(self._blocks, ...) (si ouverte)  
  └── _refresh_dashboard_stats()
```

Widgets — signaux d'intention (bottom-up)

| Signal                     | Fichier                                              | Payload                                    | Déclenché par                                     |
|----------------------------|------------------------------------------------------|--------------------------------------------|---------------------------------------------------|
| property_change_requested  | block_properties_editor.py                           | dict — {block_id, champ: valeur}           | Édition d'un champ dans l'inspecteur              |
| relative_path_changed      | block_property_widget.py                             | (block_id, container_id, relative_path)    | Modification du chemin FreeTree                   |
| blocks_changed             | free_tree_widget.py                                  | list[Block]                                | Toute mutation FreeTree (drag, rename, delete...) |
| tree_changed               | free_tree_widget.py                                  | FreeTree                                   | Structure d'arbre modifiée                        |
| block_selected             | free_tree_widget.py, workspace_tree_panel_widget.py  | (Block, container_id)                      | Sélection dans l'arbre                            |
| node_selected              | workspace_graph_widget.py                            | str (block_id)                             | Sélection dans le graphe                          |
| link_create_requested      | workspace_graph_widget.py                            | (container_id, src_id, tgt_id, port, name) | Drag de connexion dans le graphe                  |
| link_delete_requested      | workspace_graph_widget.py                            | (container_id, src_id, tgt_id, port, name) | Suppression d'un lien                             |
| graph_block_move_requested | workspace_graph_widget.py                            | (container_id, block_id, x, y)             | Déplacement d'un nœud                             |
| block_selected             | carousel_3d_widget.py, horizontal_carousel_widget.py | Block                                      | Clic sur une vignette                             |
| block_activated            | carousel_3d_widget.py, horizontal_carousel_widget.py | Block                                      | Double-clic (ouvre detail)                        |
| blocks_changed             | thumbnail_list_window.py                             | list[Block]                                | Modification dans la fenêtre vignettes            |
| shot_update_requested      | story_shot_workspace_widget.py                       | dict                                       | Mise à jour d'un shot                             |

Panels — signaux relayés (bottom-up vers MainWindow)

| Signal                      | Panel                                        | Payload                              | Handler MainWindow                          |
|-----------------------------|----------------------------------------------|--------------------------------------|---------------------------------------------|
| block_update_requested      | CharacterWorkspacePanel, StoryWorkspacePanel | dict — champs à mettre à jour        | _update_character_block_from_workspace()    |
| relative_path_changed       | CharacterWorkspacePanel, StoryWorkspacePanel | (block_id, container_id, path)       | _on_character_block_relative_path_changed() |
| character_create_requested  | CharacterWorkspacePanel                      | str (container_id)                   | _on_character_create_requested()            |
| character_update_requested  | CharacterWorkspacePanel                      | dict                                 | _update_character_from_workspace()          |
| graph_link_create_requested | CharacterWorkspacePanel, StoryWorkspacePanel | (container_id, src, tgt, port, name) | _on_graph_link_create_requested()           |
| graph_link_delete_requested | CharacterWorkspacePanel, StoryWorkspacePanel | (container_id, src, tgt, port, name) | _on_graph_link_delete_requested()           |
| graph_block_move_requested  | CharacterWorkspacePanel, StoryWorkspacePanel | (container_id, block_id, x, y)       | _on_graph_block_move_requested()            |
| new_project_requested       | ProjectWorkspacePanel                        | —                                    | _on_new_project_requested()                 |
| open_project_requested      | ProjectWorkspacePanel                        | —                                    | _on_open_project_requested()                |
| save_requested              | ProjectWorkspacePanel                        | dict (metadata)                      | _on_project_save_requested()                |
| blocks_changed              | FreeTreeWindow, ThumbnailListWindow          | list[Block]                          | _persist_project_blocks()                   |

Signaux d'infrastructure

| Signal               | Fichier                      | Usage                                    |
|----------------------|------------------------------|------------------------------------------|
| navigation_requested | sidebar_menu.py              | Changement de workspace actif            |
| theme_changed        | settings_workspace_widget.py | Rechargement du QSS                      |
| mode_changed         | mode_switch_widget.py        | Basculement de vue (tree / graph / grid) |
| log_entry            | QLogHandler (🔗 planifié)     | Alimentation de EventLogPanel            |

Contrat pour tout nouveau widget affichant des Blocks

```
class MonNouveauWidget(QWidget):
    # 1. Déclarer les signaux d'intention AVANT __init__
    block_update_requested = Signal(dict)    # si le widget peut modifier un Block
    block_selected = Signal(object)         # si le widget permet une sélection

    def __init__(self, parent=None):
        super().__init__(parent)
        self._blocks: list[Block] = []
        self._project_root: str = ""

    # 2. set_blocks() OBLIGATOIRE – point d'entrée unique pour les données
    def set_blocks(self, blocks: list[Block], project_root: str = "") -> None:
        self._blocks = blocks                # remplace toujours – jamais de merge
        self._project_root = project_root
        self._refresh()                      # redessine depuis _blocks

    def _refresh(self) -> None:
        # Filtrer, trier, afficher – tout depuis self._blocks
        ...

    # 3. Émettre un signal d'intention – jamais modifier self._blocks directement
    def _on_user_edit(self, block_id: str, new_name: str) -> None:
        self.block_update_requested.emit({"block_id": block_id, "name": new_name})
        # Ne PAS modifier self._blocks ici – attendre set_blocks() de retour
```

Règles de câblage dans MainWindow

```
# Dans MainWindow.__init__ – câbler TOUS les signaux du nouveau panel
self._mon_panel.block_update_requested.connect(self._on_block_update_from_mon_panel)
self._mon_panel.block_selected.connect(self._on_block_selected)

# Dans _refresh_project_workspace() – ajouter le nouveau panel
self._mon_panel.set_blocks(self._blocks, project_root=self._project_root)
```

**Ne jamais câbler un signal de widget directement vers un autre widget.**  
Tout passe par `MainWindow` — c'est lui qui détient `self._blocks` et orchestre la redistribution après chaque mutation.

Gestion des erreurs et journal d'événements

Exceptions du domaine

Définies dans `src/domain/exceptions.py`. Toutes héritent de `DomainError`.

| Exception       | Levée quand                                                        |
|-----------------|--------------------------------------------------------------------|
| DomainError     | Classe de base — toute erreur métier                               |
| NotFoundError   | Un Block demandé n'existe pas dans le <code>BlockRepository</code> |
| ValidationError | Une règle métier est violée (hiérarchie, type, accès...)           |

Règle de gestion dans `UseCaseService` :

Laisser remonter `ValidationError` et `NotFoundError` — l'UI les intercepte et les affiche.  
Ne jamais avaler silencieusement (`except Exception: pass`) sauf cas de dégradation explicitement documentée.

État actuel — dette de logging

`main.py` ne configure aucun logger Python. Les `except Exception` de `main_window.py` envoient des messages éphémères vers `_set_workspace_link_feedback()` — un feedback contextuel par workspace, non persisté, non horodaté, non structuré.

#### Ce qui manque :

- Pas de `logging.basicConfig` dans `main.py`
- Pas de `logger = logging.getLogger(__name__)` dans les modules
- Pas de fichier de log sur disque
- Pas de zone UI listant l'historique des événements et erreurs

### Architecture cible — Logging + EventLogPanel

#### 1. Configuration du logger en entrée d'application

```
# src/main.py — à ajouter
import logging
import sys

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s - %(message)s",
    handlers=[
        logging.StreamHandler(sys.stdout),          # console dev
        logging.FileHandler("logs/sbc.log"),        # fichier rotatif 
    ],
)
```

#### 2. Logger par module

Chaque module déclare son propre logger — pas de logger global partagé.

```
# Dans chaque fichier service / application / UI
import logging
logger = logging.getLogger(__name__)

# Usage
logger.info("Block créé : %s", block.id)
logger.warning("Block introuvable : %s — fallback projet_root", block_id)
logger.error("Erreur persistance blocks.json : %s", exc, exc_info=True)
```

#### 3. Handler Qt — pont vers l'UI

Un `QLogHandler` (subclass de `logging.Handler`) capte les entrées log et émet un signal Qt pour alimenter l'`EventLogPanel` sans couplage direct.

```
# src/UI/windows/app_log_handler.py (à créer)
import logging
from PySide6.QtCore import QObject, Signal

class QLogHandler(QObject, logging.Handler):
    log_entry = Signal(int, str, str)  # (level, logger_name, message)

    def emit(self, record: logging.LogRecord) -> None:
        self.log_entry.emit(record.levelno, record.name, self.format(record))
```

#### 4. EventLogPanel — zone UI persistante

Composant à créer dans `UI/Widgets/event_log_panel.py`.

- Widget collapsible en bas de `MainWindow` (barre escamotable)
- Affiche un journal scrollable horodaté
- Code couleur par niveau de sévérité :

| Niveau | Couleur    | Exemples                                  |
|--------|------------|-------------------------------------------|
| INFO   | Gris clair | Block créé, projet chargé, import terminé |

| Niveau  | Couleur | Exemples                                                  |
|---------|---------|-----------------------------------------------------------|
| WARNING | Orange  | Block introuvable (fallback), lien cassé, LOCATION orphan |
| ERROR   | Rouge   | Échec de persistance, exception inattendue                |

- Bouton **Effacer** (clear log)
- Bouton **Copier** (copier tout le log dans le presse-papiers pour le support)
- 🗑️ Badge sur l'icône du journal si erreurs non vues

| < Journal d'événements | [Effacer] [Copier]                   |
|------------------------|--------------------------------------|
| 10:42:01 INFO          | Projet "Mon Film" chargé (42 blocks) |
| 10:42:03 INFO          | Block créé : blk_image_a3f7c9d1e2b0  |
| 10:43:11 ⚠️ WARN       | Block blk_xxx introuvable, fallback  |
| 10:44:55 * ERROR       | Persistance échouée blocks.json      |

## 5. Intégration dans MainWindow

```
# Câblage dans MainWindow.__init__
self._log_handler = QLogHandler()
logging.getLogger().addHandler(self._log_handler)
self._log_handler.log_entry.connect(self._event_log_panel.append_entry)
```

## 6. Convention de gestion dans l'UI

```
# Pattern à respecter dans MainWindow et les panels
try:
    use_case.create_block(...)
except ValidationError as exc:
    logger.warning("ValidationError - create_block : %s", exc)
    self._show_user_feedback(str(exc)) # message inline court
except Exception as exc:
    logger.error("Erreur inattendue - create_block", exc_info=True)
    self._show_user_feedback("Erreur interne - voir le journal.")
```

`_show_user_feedback()` = feedback éphémère contextuel (existant).

`EventLogPanel` = journal persistant horodaté (à créer).

Les deux coexistent : le feedback est pour l'action immédiate, le journal pour l'historique.

## Principes techniques

- **UI** : PySide6 — desktop natif, macOS + Windows
- **Modèle** : un seul type d'entité **Block** — typage par **BlockType** + **profile**
- **Accès au contenu** : toujours via `as_media()` / `as_container()` en lecture ; écriture directe dans `block.content[...]` avec les clés canoniques
- **Stockage** : `blocks.json` partitionné par workspace ; un `project.json` racine
- **Même format** projet et librairie — seul `kind` diffère
- **Écriture atomique** 🗑️ : `.tmp` + renommage — aucune corruption possible
- **IDs immuables** : `blk_<uuid_hex12>` — jamais le nom lisible comme identifiant
- **Couche repository** : **BlockRepository** (dict RAM) isole l'app du format — migration SQLite/graphDB possible
- **Façade unique** : **UseCaseService** — l'UI n'appelle jamais **BlockService** directement
- **Clés API** : 🗑️ trousseau système uniquement — jamais dans un JSON
- **Offline** : fonctionnel hors connexion pour l'organisation ; IA optionnelle en ligne

## Règles de codage

1. **Import du domaine** : toujours depuis `domain` (le package), pas depuis `domain.models` directement.

```
from domain import Block, BlockType, BlockDomain, MediaContent, ContainerContent
```

2. **Lecture de contenu** : utiliser les accesseurs, jamais `block.content.get("key")` dans l'UI ou les services.

```
# ✔  
role = block.as_container().workspace_role  
path = block.as_media().storage_path  
# ✖  
role = block.content.get("workspace_role", "")
```

3. **Écriture de contenu** : écrire directement dans le dict avec les clés canoniques documentées dans `block_content.py`. Ne jamais inventer une clé qui n'y figure pas.

```
block.content["workspace_role"] = "story_root"  
block.content["storage_path"] = str(destination)
```

*(Cette asymétrie lecture/écriture est une dette de refactoring — voir la section "Contenu typé".)*

4. **Création de Block** : toujours via `UseCaseService.create_block()` ou `create_block_in_container()`.

5. **Nouveau profil** : l'ajouter dans le set `PROFILES` de `domain/models.py`.

6. **Nouveaux champs de contenu** : les documenter dans `domain/block_content.py` avec les clés canoniques.

7. **IDs** : format `blk_<uuid4().hex[:12]>` — utiliser `UseCaseService.create_block()` qui génère automatiquement.